⊛ ACID

⊛ Consistency

Rules & Constraints

Eg. $\begin{cases} A \rightarrow -100 \\ B \rightarrow +100 \end{cases}$ If A is debited ₹100, B must be credited ₹10 =.

Q How DB knows that ₹100 deduction & consequent addition makes it consistent? =.

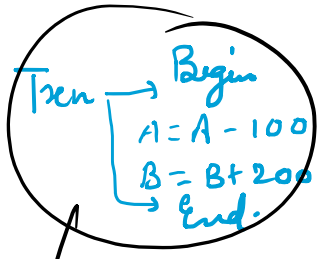
Lets say, there is a Bug in the app code.

↓

always deducts ₹10 more.

↓

Will DB reject this?



DB can't reject it in normal scenarios.

Note "Credit & Debit must match"

⇓

Appl^{n} level Consistency.

guarantee provided by appl^{n}

Data Corruption ≠ ~~Data~~ DB Inconsistency.

Data Inconsistency.

Here, we're talking about DB Consistency.

(*) A → -100
B → +500

→ Was data inconsistent → YES
↳ who broke consistency
↳ was it DB → NO
↳ " " Application → YES

{Consistency are Rules/Constraints guaranteed by DB}

Eg.
```
Create table users(
    id int,
    name varchar,
    age int check(age >= 20),
);
```

Now its guaranteed by DB (schema) that a value < 20 will be rejected auto by DB.

Q If we want Unique emails.

OR

either the Appl^{n} controls this

↓

check if DB has any record w/ the email

YES → Reject

NO → create

=

||

App level Consistency

DB controls this

↓

use UNIQUE constraint w/ email col., when defining the schema. i.e.

||

DB level Consistency

DB is a dumb sys.

— it only respects constraints / checks configured by you on tables

(*) Atomicity

↳ Do you think std. eg. of Atomicity
Bank bal. / UPI payments

{ A −100
B +100 }

must happen in single Txn.

Had it been same table & in presence of same txn. → Yes ✓ its atomicity

BUT

Practically,
money transfers are not necessarily same DB txn.

(Note:- There is no concept of DB Level CROSS-DB txn)

9:43 pm

Atomicity → No concept of Partial Updates

Note :- Not just TRANSACTION

Eg. If I just say

```
update employees
set sal = sal + 0.1sal;
```

```
set sal = sal + 0.1*sal;
```

Behind the scenes

```
for (Employee e : employees) {
    e[sal] += 0.1*e[sal];
}
```

its row by row calculation

O(N)

Atomicity ensures either all get updated or none.

=) there is an implicit TXN scope working for every operation executed on DB.

∴ Atomicity also means if I send a query, it must execute [illegible] & not partially.

it must execute completely & not partially.

→ DB ensures either ALL or NONE.

⊛ I → Isolation

One txn. can't interfere w/ the other.

∴ One txn. runs as if its the only one

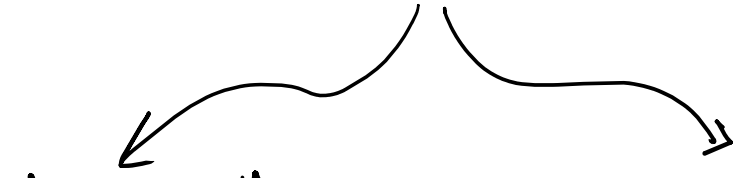
Note!- Multiple txn's can run ||ly

You can control → To what level two or more || txn. can interfere w/ each other.

You can't control 11 txn. to not run.
BUT
to what level they interfere w/ each other

Say UserA → update sal from 90k to 130k → Txn1

UserB → update sal from 90k to 100k → Txn2



Issue won't happen if they are diff. rows/record

Prob. happens if its same record

↓

final result is unpredictable

Note :- "Isolation"
↳ helps us w/ how we deal w/ multiple operations happening together on a record.

Isolation Levels

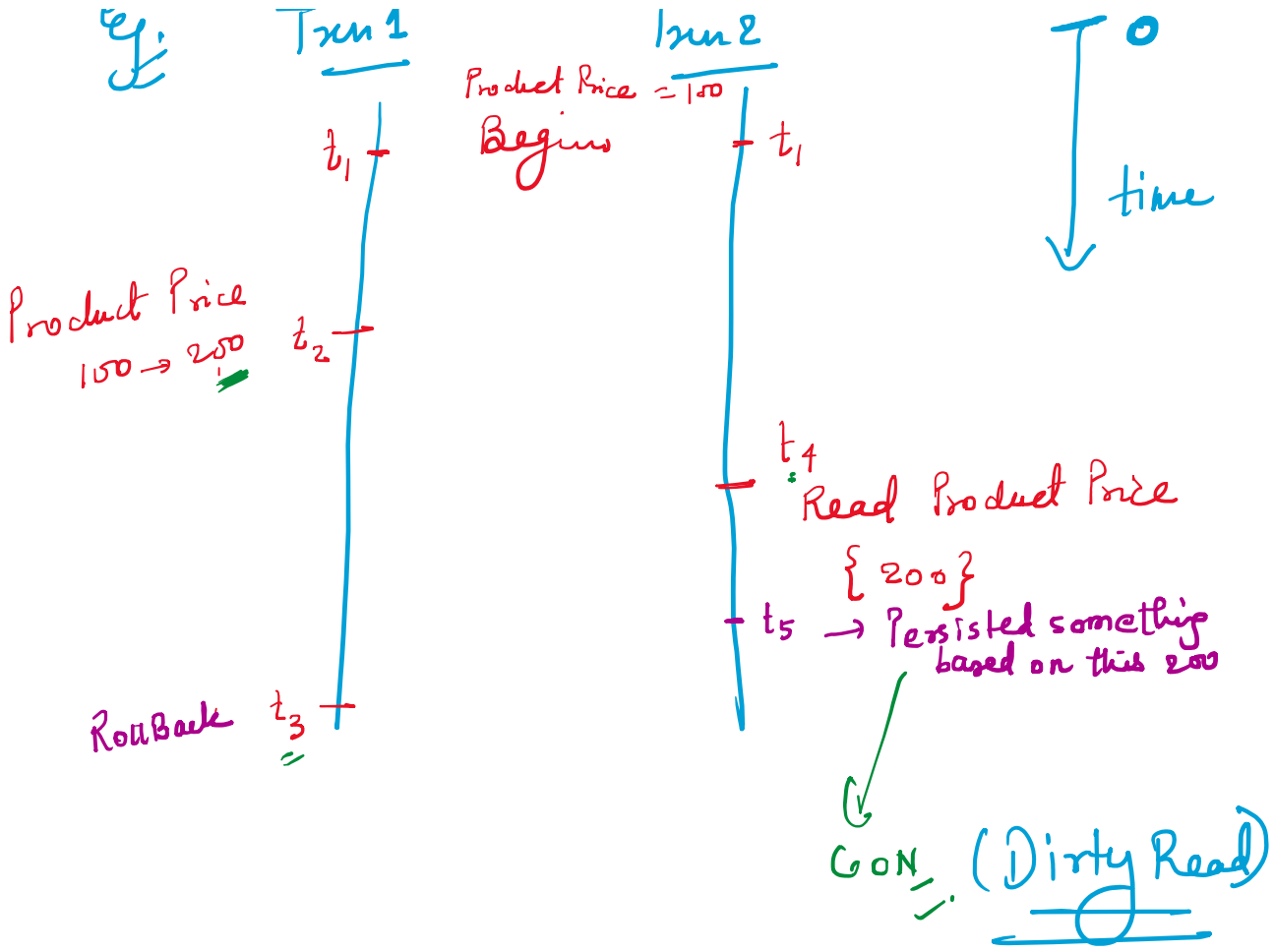
↳ define to what extent interference exists among ll^r txn.^s

a.) Read Uncommitted.

Two txn are such isolated such that they read the writes uncommitted from the other.

Eg: Txn 1 Txn 2 T^0

Product Price = 100 ₹



Use :- high traffic

B2C dashboard (buy behaviour)

↓

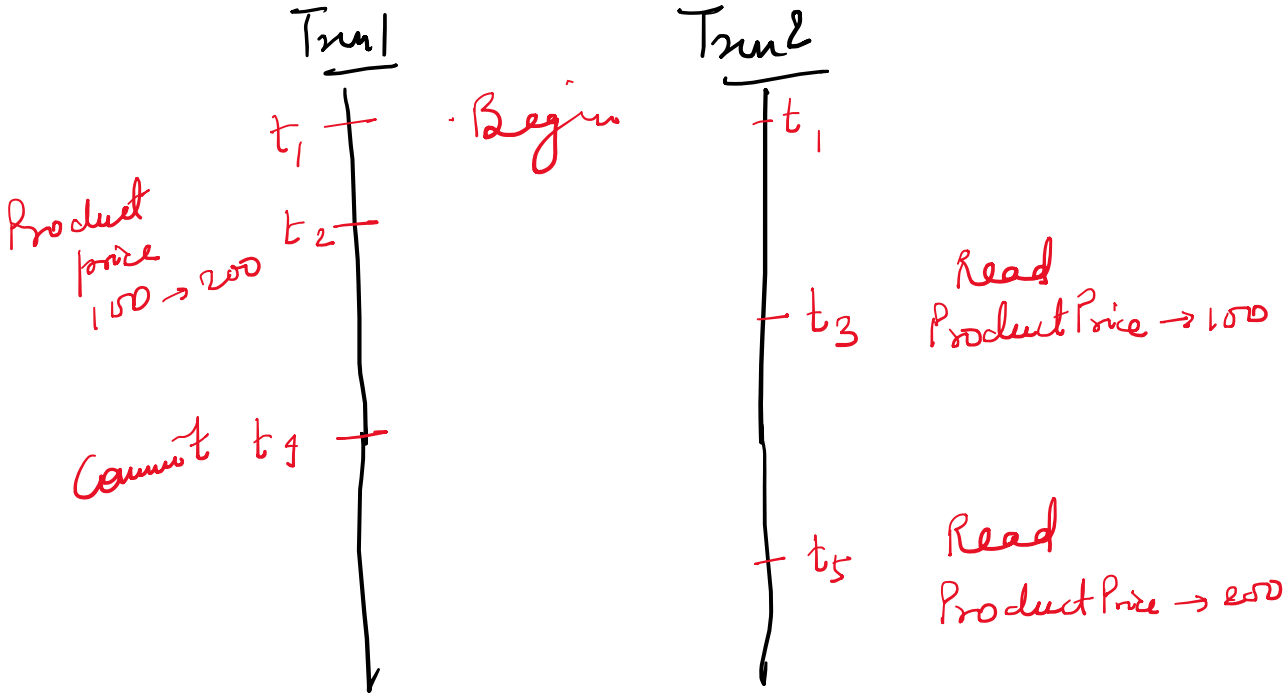
you are not interested in actual data but its trends.

alongwith able to handle high traffic

b.) Read Committed (Default Isolation level provided by DB)

As the name suggests,

As the name suggests,
only read committed data
from other txn.



Adv. Solves prev. Isolation level prob./con

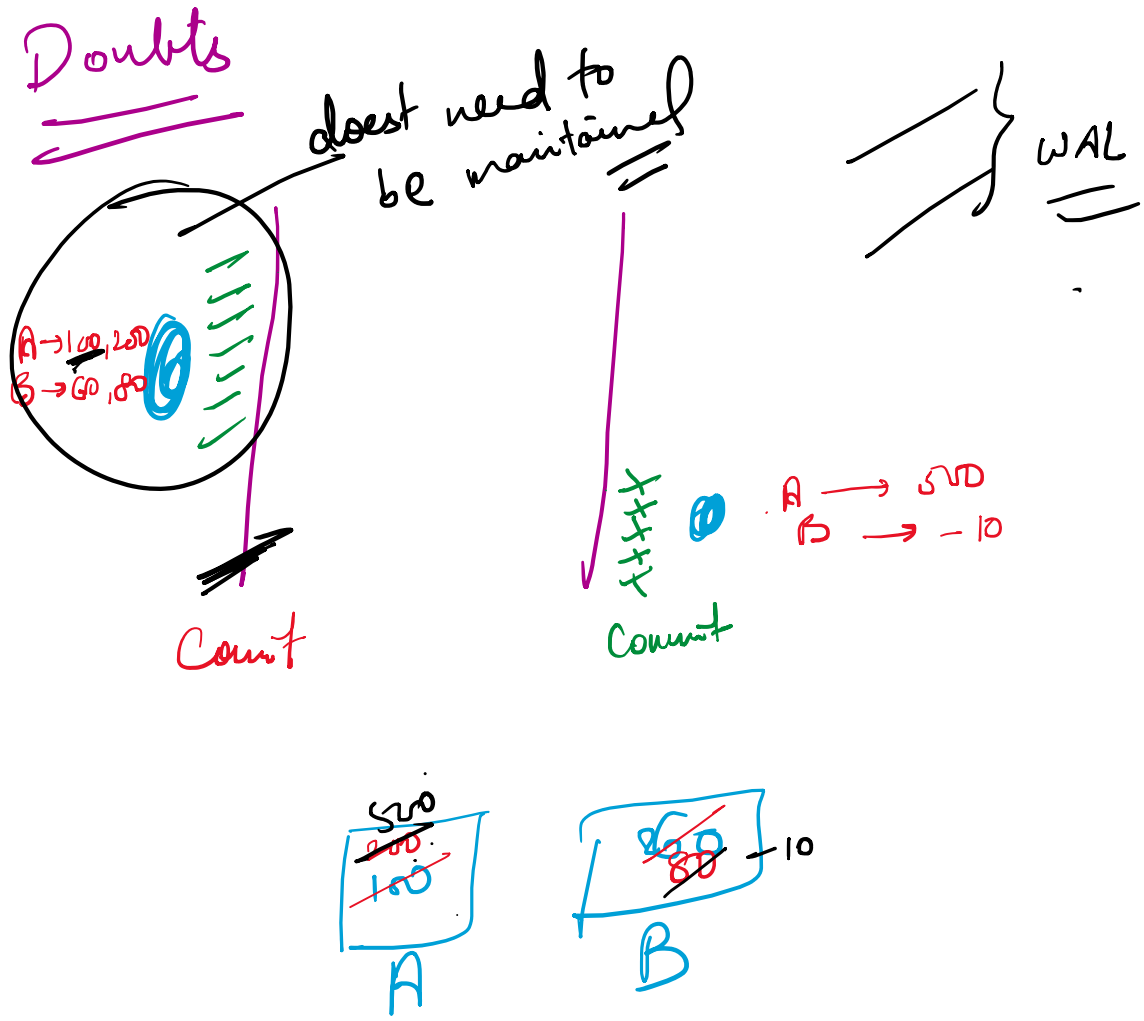
Note Uncommitted writes are being maintained somewhere else

⇓

⇒ extra overhead

⇓

∴ slower than Read Uncommitted


Doubts
doest need to be maintained
WAL
A → 100, 200
B → 60, 80
Commit
Commit
A → 500
B → -10
500
200
100
A
80
-10
B